Informe desafio 2
Ude@book

Juan Camilo Ramirez

1. **Análisis del problema.**
   Al momento de realizar el análisis es fundamental entender a fondo que se estaba solicitando en el programa, ahí radica la pregunta ¿Qué cosas existen en Ude@book? Usuarios, amigos, mensajes, álbumes, fotos, solicitudes y sugerencias. Cada uno de esos objetos tienen sus propios datos y se relacionan con los demás. Este fue el punto de partida para decidir las clases y de que se encargaría cada una.

   A. Sugerencias de clases
   1. redsocial
   2. usuario
   3. mensaje
   4. álbum
   5. fotos
   6. fecha

   B. Sugerencias de atributos
   1. redsocial
      usuarios, cantidad de usuarios, capacidad de usuarios, matriz de amigos, iteraciones actuales, memoria consumida.
   2. usuario
      nombre, fecha de nacimiento, correo, foto de perfil, contraseña, ciudad, país, fecha de inscripción, amigos, mensajes enviados, mensajes recibidos, solicitudes recibidas, álbumes.
   3. mensaje
      emisor, receptor, texto, fecha de envío, leído.
   4. álbum
      nombre, ruta, fotos, cantidad fotos, capacidad fotos, ¿privacidad?.
   5. foto
      fecha de captura, tamaño, ruta, likes.
   6. fecha
      día, mes, año

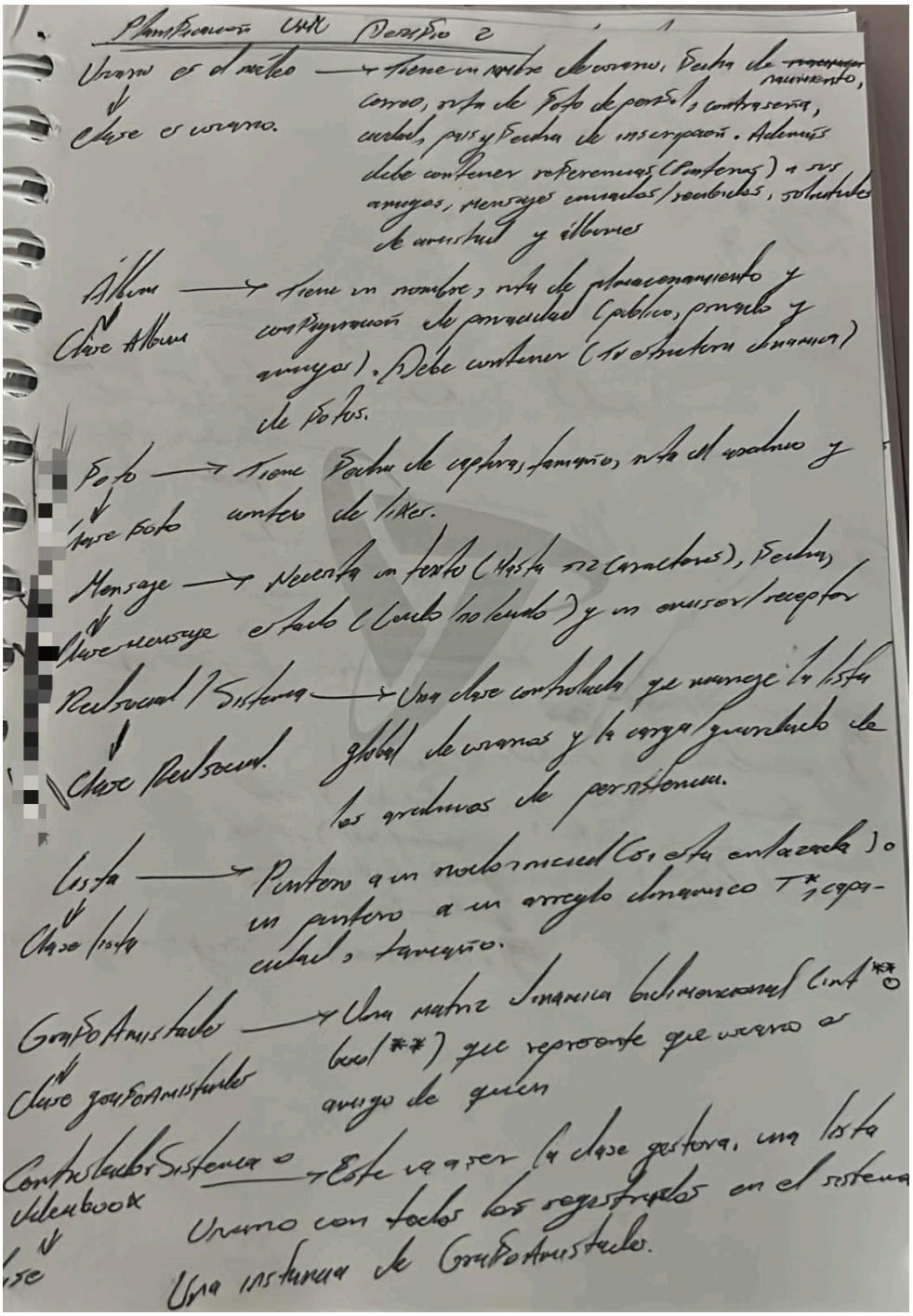
   C. Sugerencias de métodos
   1. redsocial
      - redSocial
      - cargarDatos
      - guardarDatos
      - iniciarSesion
      - buscarUsuario

- enviarSolicitud
- aceptarSolicitud
- obtenerSugerencias
- amigosEnComun
- administrarMensajes
- medirConsumoRecursos
  una posible sobrecarga se podría implementar en buscar usuario poniendo diferentes parámetros como el correo, la fecha de inscripción, etc…

2. usuario
   - agregarAmigo
   - verificarAmistad (si ya son amigos o no)
   - enviarSolicitus
   - aceptarSolicitud
   - enviarMensaje
   - recibirMensaje
   - marcarMensajeLeido
   - buscarAlbum
   - buscarMensaje
   - mostarAlbunes
     una posible sobrecarga se podría implementar en buscar mensaje filtrando ya sea por usuario o la fecha.
3. mensaje
   - marcarComoLeido
   - estaLeido
   - mostrarMensaje
   - obtenerTexto
   - obtenerFecha
   - rangoFechas
4. álbum
   - buscarFoto
   - obtenerFoto
   - mostarFoto
   - visiblePara
   - obtenerCantidadFotos
5. foto
   - darLike
   - quitarLike
   - obtenerRuta
   - obtenerFechaCaptura
   - obtenerLikes
   - mostrarRuta
6. fecha

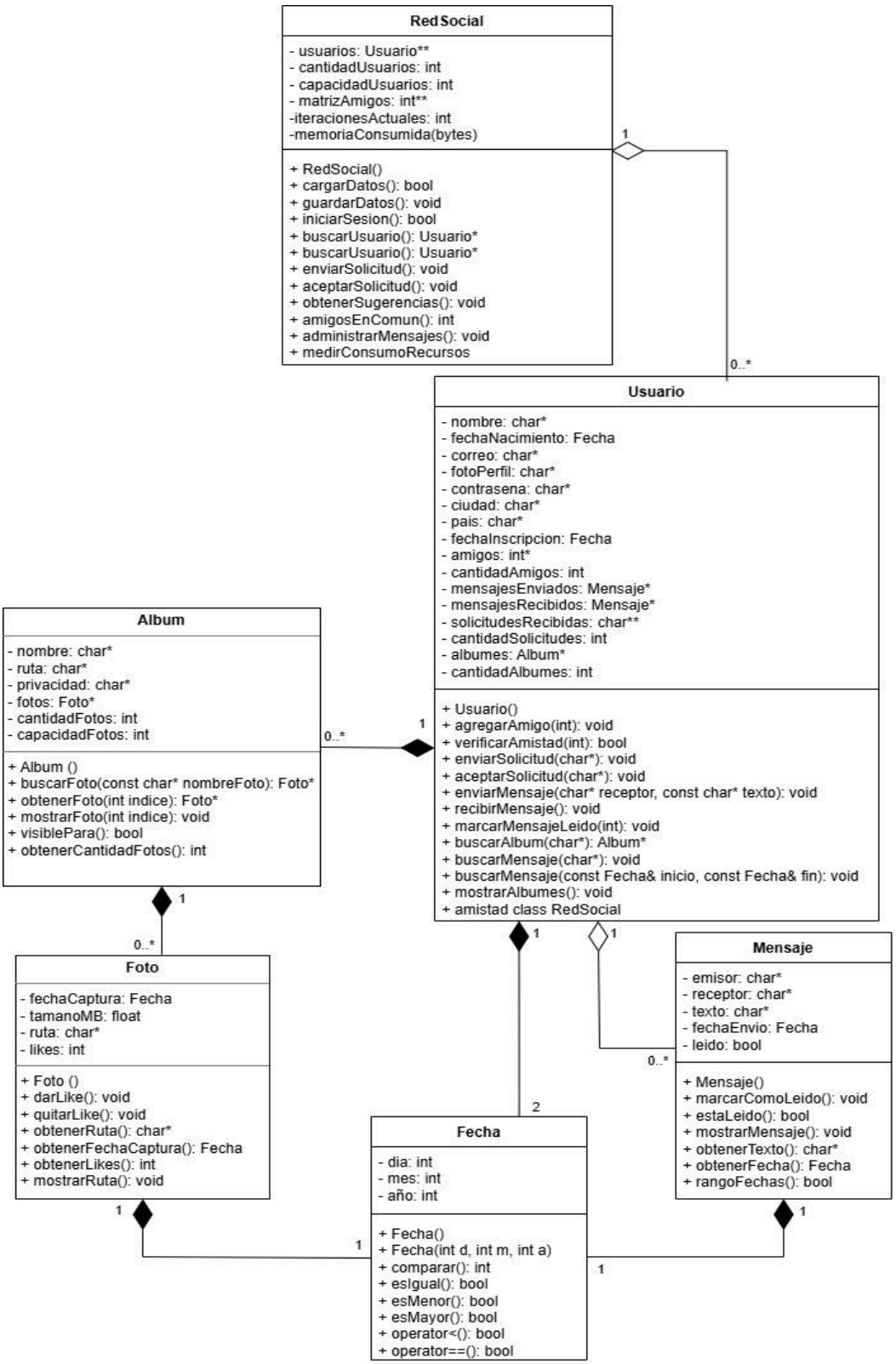
- comparar
- esIgual
- esMenor
- esMayor
- Sobrecarga de operadores

Nota: En el apartado anterior se optó por agregar “sugerencias” ya que los datos usados pueden estar sujetos a cambios al momento de realizar el programa dependiendo de las necesidades o dificultades que se presenten.



2. Diseño de la solución (Diagrama de clase)

En esta sección se presenta el diseño de la solución mediante un diagrama de clases UML, donde se describen las diferentes clases implementadas y sus respectivos atributos y métodos y cómo interactúan entre sí.



En la figura anterior se muestra el diagrama de clases propuesto, donde se observan las relaciones entre las clases y las multiplicidades que soportan las funcionalidades principales.

3. Algoritmos esenciales

a. carrusel de fotos: En este algoritmo el sistema mantiene un índice que indica la posición de la foto actual en el álbum. al presionar “siguiente” el índice incrementa. para evitar que al llegar a la última foto el índice se salga del arreglo se usó (%) con la cantidad total de fotos. Si el índice es la última foto, con la operación (índice +1) % cantidad devuelve 0 que sería la primera foto, reiniciando el recorrido. pasa lo mismo en sentido contrario cambiando el signo y así se crea el ciclo.

b. aceptar solicitud de amistad: cuando un usuario acepta una solicitud, el sistema primero agrega al emisor a la lista de amigos (verificando si hay espacio y redimensionando si es necesario). Luego, procede a eliminar la solicitud de la lista de pendientes. Para ello, se libera la memoria de la cadena en la posición específica y se recorre el arreglo desplazando cada elemento una posición hacia la izquierda. Finalmente, se reduce el contador total para evitar que la última posición quede duplicada, asegurando que el arreglo quede compacto y sin basura. Por último el usuario que envió la solicitud hace el proceso de agregar a la lista de amigos y redimensión si es necesario.

c. Redimensionamiento Dinámico de Estructuras: teniendo en cuenta a la prohibición de usar la STL no se puede usar (std::vector), para todas las listas (álbumes, amigos, mensajes) se usaron arreglos propios. El algoritmo inicia con una capacidad mínima. Antes de insertar un nuevo elemento, se verifica si la cantidad actual iguala la capacidad. Si ese es el caso, se invoca un método auxiliar que duplica la capacidad, luego solicita un nuevo bloque de memoria al sistema operativo (new), copia los datos existentes uno por uno, y libera el bloque de memoria anterior. Esto garantiza un crecimiento eficiente en memoria.

d. Constructor de Copia (Evitar doble delete): al momento de crear un objeto como un usuario a partir de otro, el sistema no puede simplemente copiar los punteros ya que serioa una copia superficial, ya que ambos apuntarían a la misma dirección de memoria y al destruirse uno, el otro quedaría corrupto (doble liberación). El algoritmo del constructor de copia reserva memoria nueva para cada atributo que sea un puntero o cadena (char*), y copia el contenido usando strlen y strcpy o los arreglos elemento por elemento. En el caso de punteros a otras entidades como los amigos, se copia la dirección de memoria, manteniendo la referencia a los objetos originales.

4. Documentación de archivos.

para este proyecto se tenía propuesto la implementación de de varios archivos de texto txt donde almacenan diferentes tipos de datos separados con “ | “. entre ellos tenemos:

- usuarios.txt:nombreUsuario|dia|mes|año|correo|rutaFoto|contraseña|ciudad|pais
- amistades.txt: usuario1|usuario2 (representa una arista en el grafo de amistades).
- mensajes.txt: emisor|receptor|texto|dia|mes|año|leido
- albumes.txt: usuario|nombreAlbum|ruta|privacidad
- fotos.txt: usuario|album|ruta|dia|mes|año|tamañoMB|likes

Nota: Pese a que se tenía planeado implementar los archivos por falta de tiempo no se logró realizar.

5. **Funcionamiento**

Por motivos de tiempo no se alcanzó a terminar el módulo de redSocial en la cual administraba el menú, las sugerencias, entre otras funciones. Para solucionar el contratiempo se diseñó un entorno de pruebas en el main que simula el flujo real de la red social.

```
#include <iostream>
#include "usuario.h"
#include "mensaje.h"
#include "album.h"
#include "foto.h"
#include "RedSocial.h"
using namespace std;

//por falta de tiempo solo probaremos los modulos ya terminados
int main(){
    cout << "PRUEBA DE MODULOS - UDE@BOOK" << endl;

    cout << "1. MODULO FECHA" << endl;
    Fecha hoy(17, 10, 2026);
    Fecha fnac(1, 1, 2001);
    Fecha finsc(1, 1, 2026);
    cout<< "Fecha de hoy: " <<hoy.getDia()<< "/"<<hoy.getMes() <<"/" << ho
    cout<< "Fecha nacimiento: "<< fnac.getDia()<<"/"<< fnac.getMes()<<"/"
    cout<< endl;

    cout<< "2. MODULO USUARIO Y AMIGOS"<< endl;
    usuario* u1 = new usuario("juanito", fnac, "juanito@udea.edu.co", "/us
    usuario* u2 = new usuario("maria", fnac, "maria@udea.edu.co", "/users/
    usuario* u3 = new usuario("carlos", fnac, "carlos@udea.edu.co", "/user

    cout << "Se crearon los usuarios: "<< u1->obtenerNombre()<<", "<< u2->

    // Haciendolos amigos
    u1->agregarAmigo(u2);
    u2->agregarAmigo(u1);
    cout<< "Ahora son amigos."<< endl;
    cout<< "Amigos de "<< u1->obtenerNombre()<< ":" << endl;
```

```
cout<< "Ahora son amigos."<< endl;
cout<< "Amigos de "<< u1->obtenerNombre()<< ":" << endl;
u1->mostrarAmigos();
cout << endl;

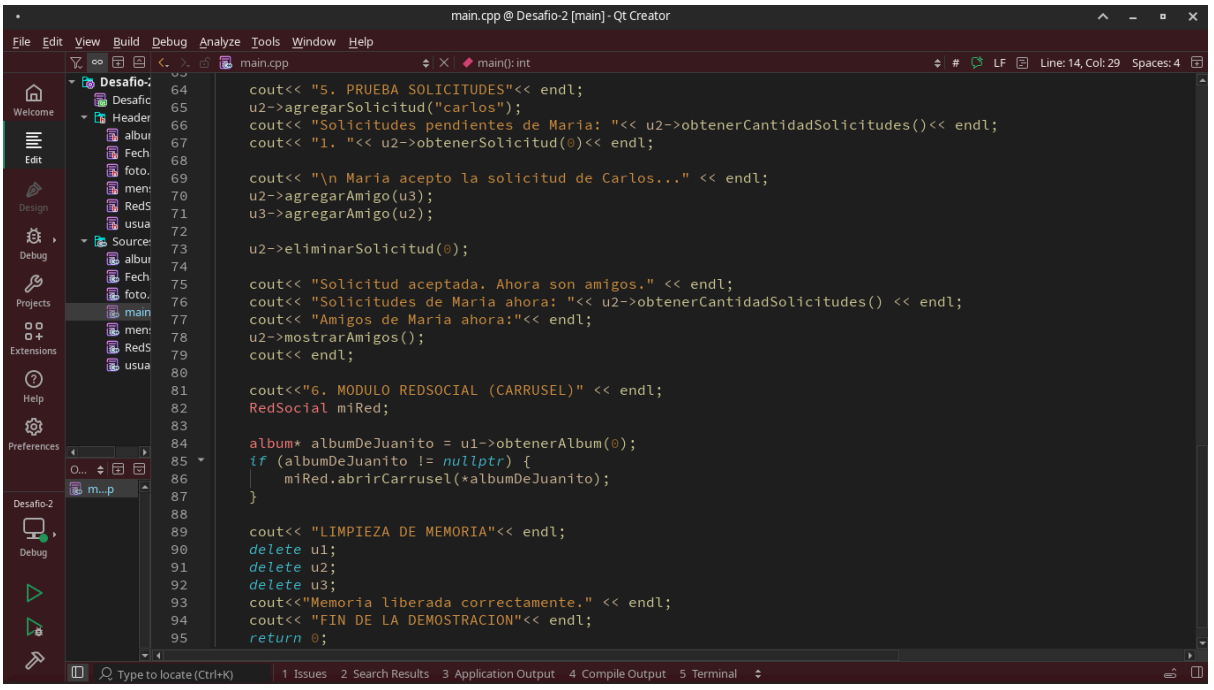
cout<< "3. MODULO ALBUM Y FOTOS"<< endl;
album a1("Cartagena2026", "/users/.../albums/Cartagena/", "amigos");

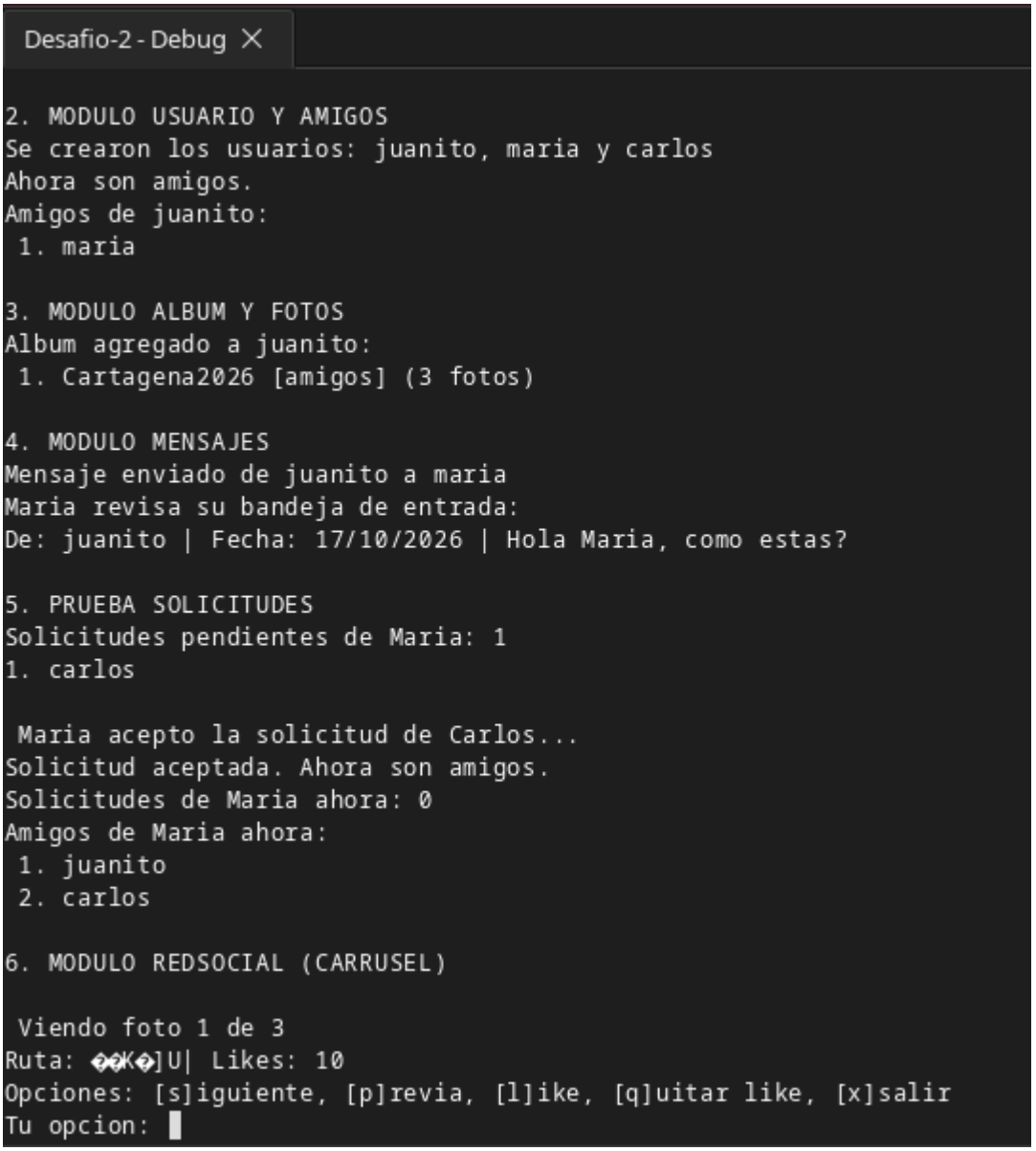
foto f1(hoy, 2.5f, "/users/.../Cartagena/Malecon.png", 10);
foto f2(hoy, 3.1f, "/users/.../Cartagena/Bocagrande.png", 20);
foto f3(hoy, 1.8f, "/users/.../Cartagena/Castillo.png", 15);
a1.agregarFoto(f1);
a1.agregarFoto(f2);
a1.agregarFoto(f3);

u1->agregarAlbum(a1);
cout<<"Album agregado a "<< u1->obtenerNombre()<<":"<< endl;
u1->mostrarAlbumes();
cout << endl;

cout<< "4. MODULO MENSAJES"<< endl;
mensaje m1("juanito", "maria", "Hola Maria, como estas?", hoy);
u1->agregarMensajeEnviado(m1);
u2->agregarMensajeRecibido(m1);
cout<<"Mensaje enviado de "<< u1->obtenerNombre()<< " a "<< u2->obtenerNombre()

cout << "Maria revisa su bandeja de entrada:" << endl;
int cant = u2->obtenerCantidadMensajesRecibidos();
for(int i = 0; i < cant; i++) {
    u2->obtenerMensajeRecibido(i)->mostrarMensaje();
}
cout<< endl;
```





6. Dificultades

Durante este desafío las dificultades que surgieron fueron principalmente gestión de tiempo (bastante tiempo invertido en investigación), problemas en el trabajo grupal, problemas de tipeo que consumia bastante tiempo el solucionarlo y problemas en el módulo redSocial que no se pudo solucionar.



7. Conclusión

En conclusión, la solución que se planteó cumple en gran medida con objetivos lógicos de un potencial cliente para la red social Ude@Book, respetando las

restricciones de diseño impuestas. Además se logró construir una capa lógica funcional que soporta la interacción entre usuarios, la navegación circular de fotografías y la gestión de estados de amistad. Si bien la interfaz gráfica y la persistencia en archivos quedaron como líneas de trabajo futuro por limitaciones de tiempo, el núcleo del sistema demostró ser operativo mediante pruebas de simulación. Este proyecto dejó como enseñanza principal que el dominio de los punteros y la memoria dinámica es indispensable para construir sistemas escalables, y que la organización modular en archivos (.h y .cpp) facilita enormemente la depuración y el mantenimiento del código.